SoC 101 | 2026-1

Session4 : Combinational Logic Blocks(1)

Agenda

- Basic Combinational Blocks(1)
  - Multiplexer & Demultiplexer
  - Encoder & Decoder
- LABS

SoC 기본 조합논리

- 조합회로 (combinational logic)
  - 논리 게이트들의 집합으로 구성되며, 회로 구성과 현재의 입력에 의해 회로의 출력 값이 결정되는 회로
  - 저장소자 (래치, 플립플롭)이 포함되지 않은 회로

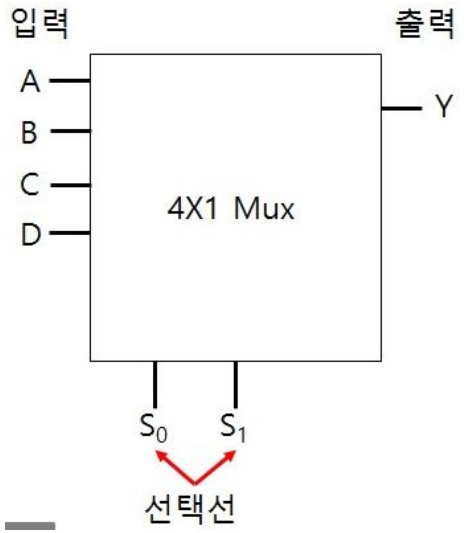
[표 10-1] 조합회로의 형태와 모델링에 사용되는 Verilog HDL 구문

조합회로의 형태	조합회로 설계에 사용되는 Verilog HDL 구문
• 기본 논리 게이트 • 멀티플렉서 • 인코더, 디코더 • 랜덤 로직 • 가산기/감산기 • 비교기 • ALU • lookup table	• 게이트 프리미티브 • 연속 할당문(assign 문) • 행위수준 모델링 (if 문, case 문, for 문, repeat 문, while 문) • 함수 및 태스크 (시간 또는 이벤트 제어를 갖지 못함) • 모듈 인스턴스

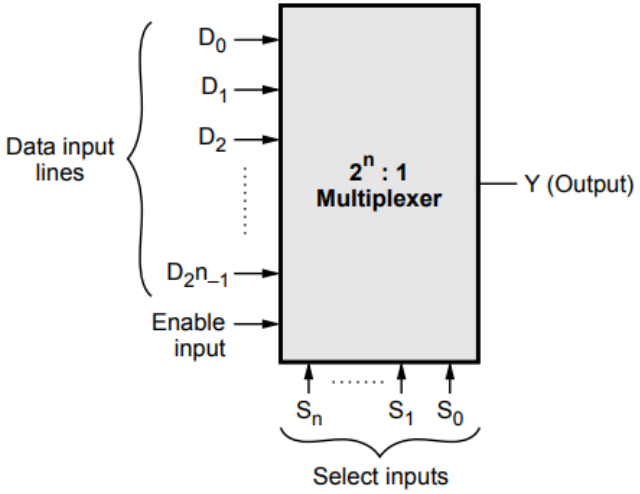
1. Multiplerxer

Multiplexer

- 멀티플렉서 (Multiplexer, MUX)
  - 여러 개의 입력 중 하나를 선택하여 단일 출력으로 전달하는 장치
  - 데이터 선택기(Data Selector)
  - 데이터 입력: 2^n개의 데이터 입력 라인 (D0, D1, D2, ...)
  - 선택 라인: n개의 선택 라인 (S0, S1, ..., Sn-1)
  - 출력: 하나의 출력 라인 (Y)



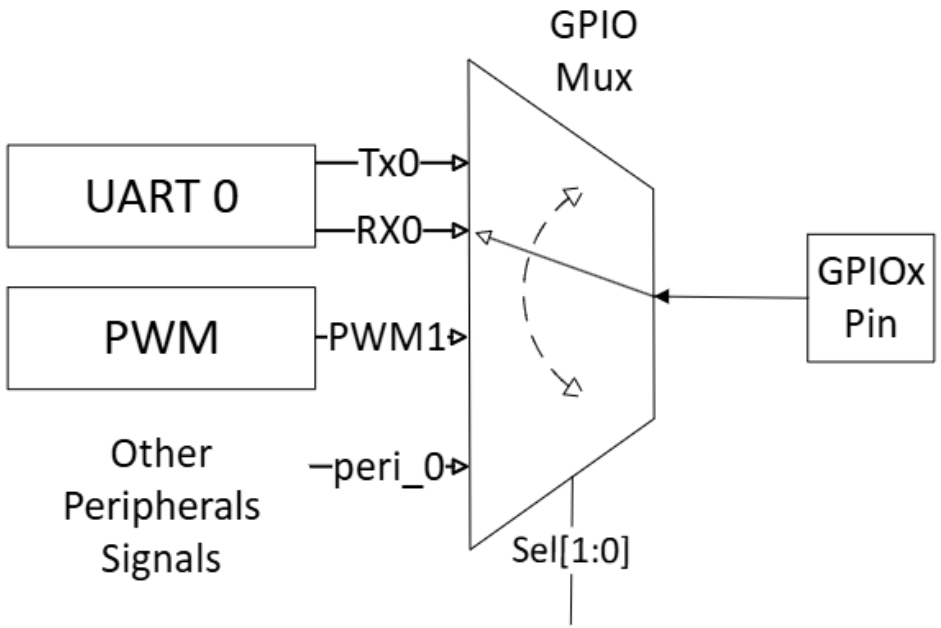
S_0	S_1	Y
0	0	A
0	1	B
1	0	C
1	1	D



(a) Block diagram of $2^n : 1$ multiplexer

LAB : Specification

- GPIO Multiplexer Specification



LAB : DUT

```
module mux(
    // port list and declaration
    input wire [1:0] sel,        // select signal
    input wire tx0,              // UART0 Tx
    input wire rx0,              // UART0 Rx
    input wire pwm1,             // PWM
    input wire other_signal,     // other peripheral signal
    output reg gpio_pin          // GPIO Output Pin
    );



    // behavioral modeling
    always @(*) begin
        case (sel)
            2'b00: gpio_pin = tx0;          // Select UART0 Tx
            2'b01: gpio_pin = rx0;          // Select UART0 Rx
            2'b10: gpio_pin = pwm1;         // Select PWM
            2'b11: gpio_pin = other_signal; // Select Ohter Peripheral Signal
            default: gpio_pin = 1'bz;       // Default: High-Z(not connected)
        endcase
    end

endmodule
```

LAB : Testbench

```
module tb_mux(
 // Test Bench
    );

    // signal list
    reg [1:0] sel;
    reg tx0;
    reg rx0;
    reg pwm1;
    reg other_signal;
    wire gpio_pin;

    // DUT Instantiation
    mux dut (
        .sel(sel),
        .tx0(tx0),
        .rx0(rx0),
        .pwm1(pwm1),
        .other_signal(other_signal),
        .gpio_pin(gpio_pin)
    );
```

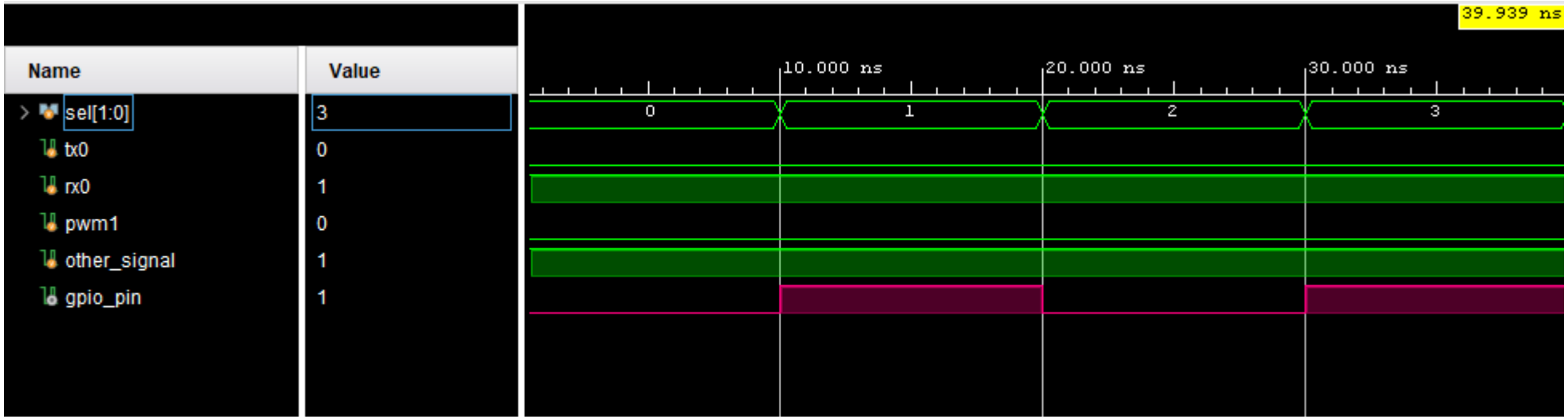
LAB : Testbench

```
    // Inject Stimulus
    initial begin
        // init
        tx0 = 1'b0; rx0 = 1'b1; pwm1 = 1'b0; other_signal = 1'b1;

        // Inject Stimulus
        sel = 2'b00; #10; // select tx0 -> gpio_pin = tx0 (0)
        sel = 2'b01; #10; // select rx0 -> gpio_pin = rx0 (1)
        sel = 2'b10; #10; // select pwm1 -> gpio_pin = pwm1 (0)
        sel = 2'b11; #10; // select other_signal -> gpio_pin = other_signal (1)

        $finish;
    end

endmodule
```

LAB : Multiplexer

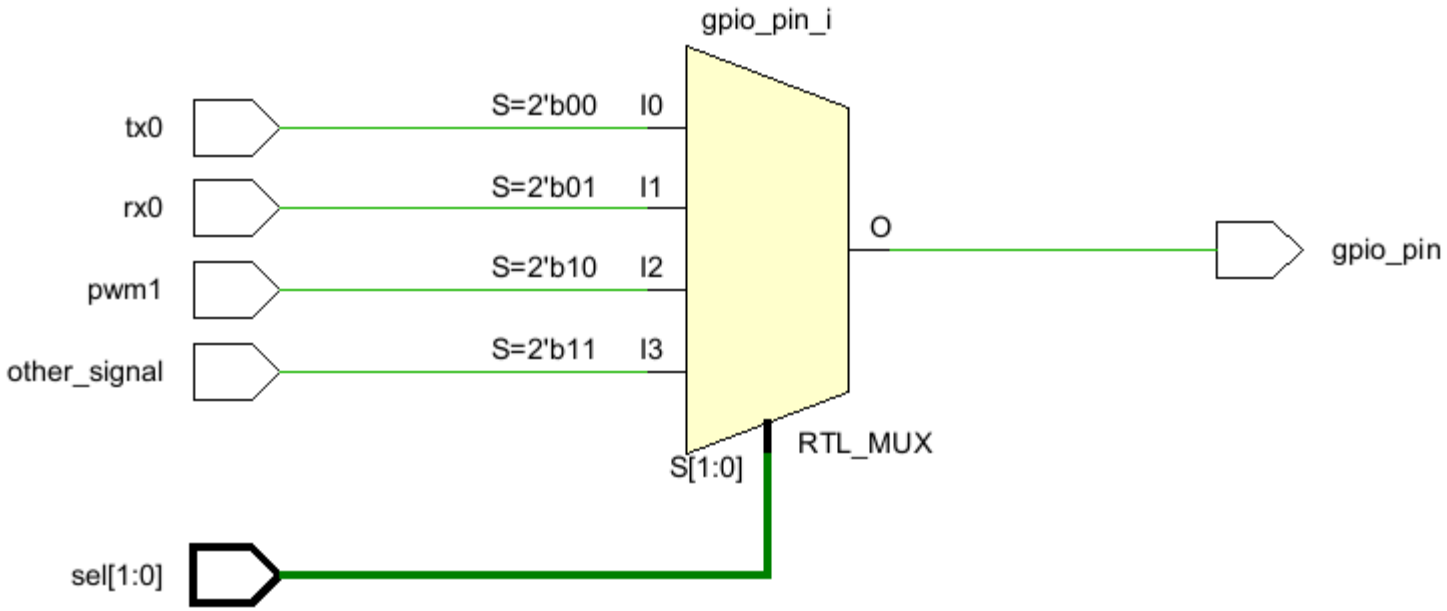
- Wave Form

sel=00 -> gpio_pin=0 (tx0)
sel=01 -> gpio_pin=1 (rx0)
sel=10 -> gpio_pin=0 (pwm1)
sel=11 -> gpio_pin=1 (other_signal)



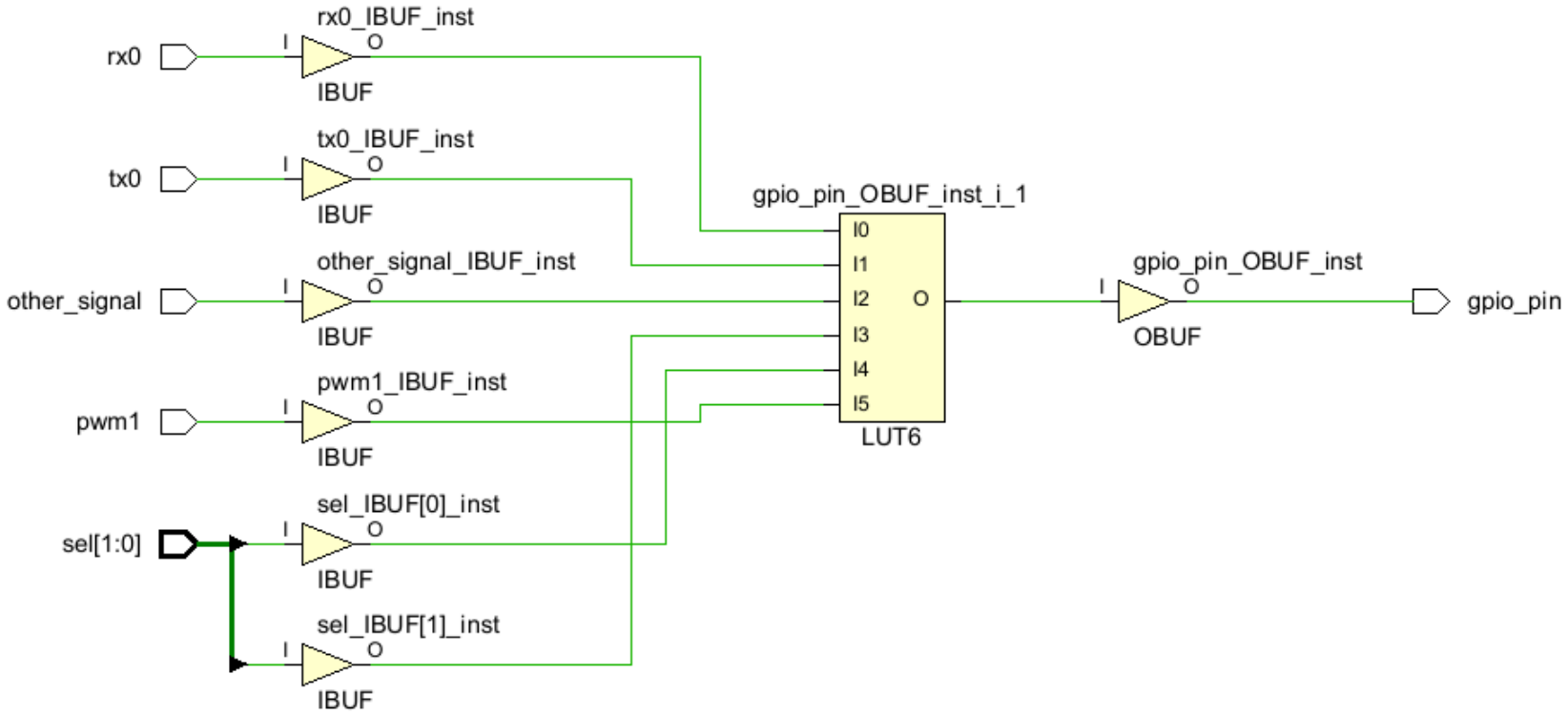
LAB : Multiplexer

- RTL Schematic



LAB : Multiplexer

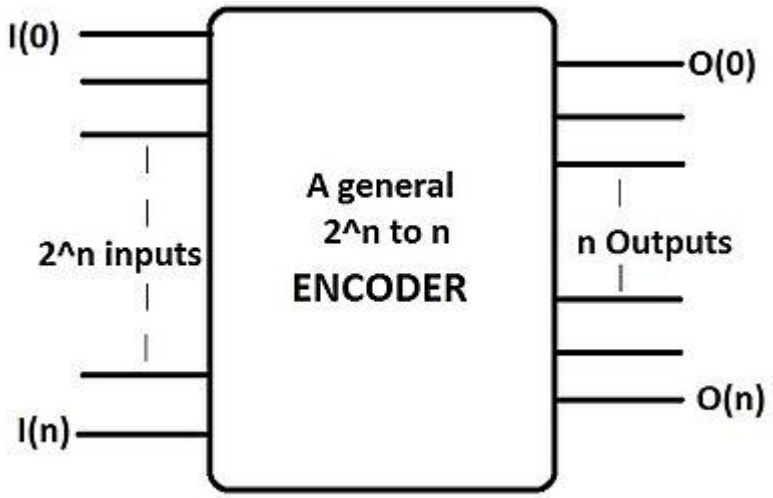
- Synthesized Schematic



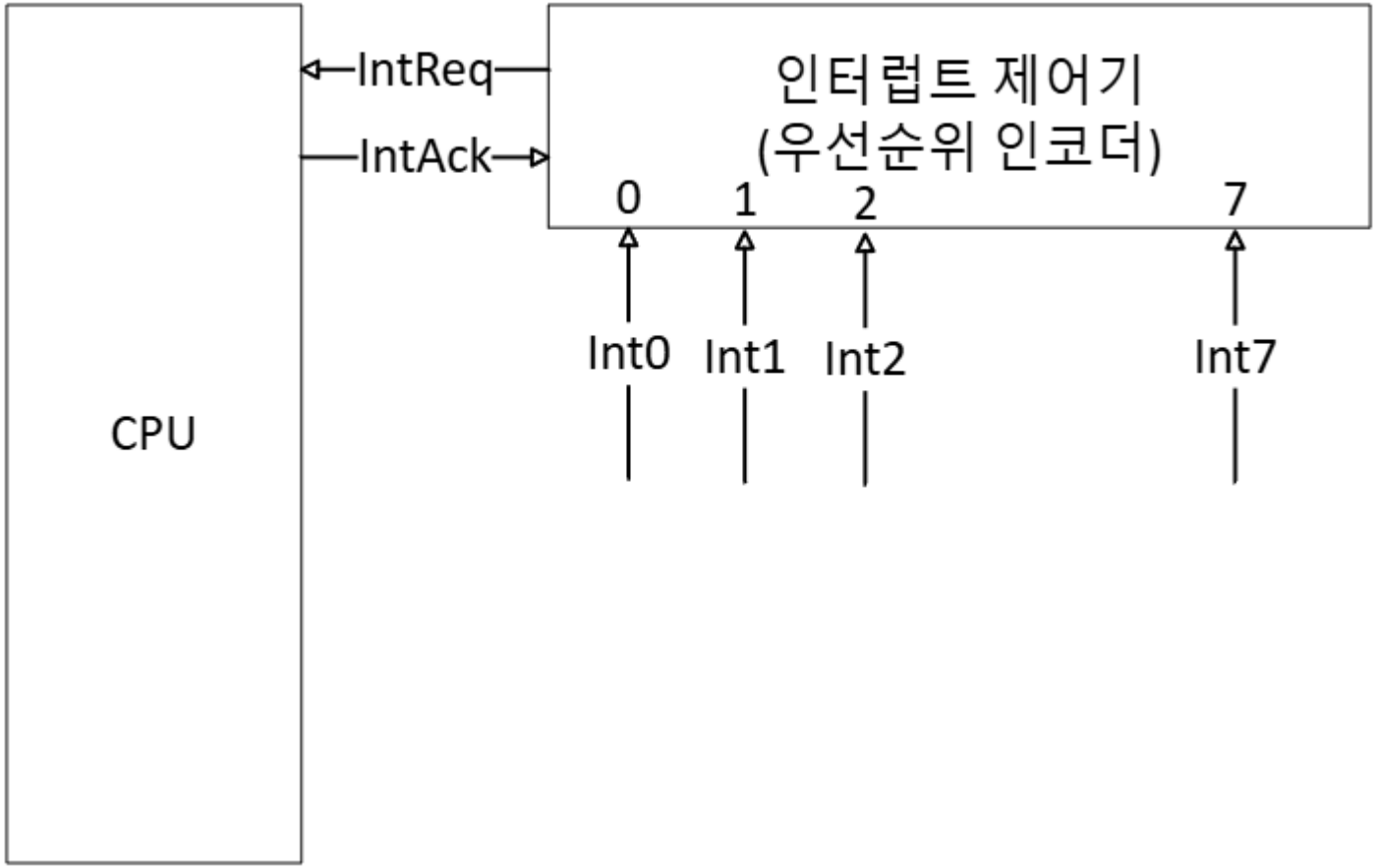
2. Encoder

Encoder

- 인코더 (Encoder)
  - 인코더는 사람이 이해하기 쉬운 정보를 기계가 처리할 수 있는 코드화된 형식으로 변환하는 회로.
  - 2^n개의 입력 라인을 n개의 출력 라인으로 변환(데이터를 더 작은 비트로 변환)
  - 일반적으로 한 번에 하나의 입력만 활성화.
  - 주요 유형:
    - 4-to-2 인코더
    - 8-to-3 인코더 (8진 인코더)
    - 10진수-to-BCD 인코더
  - 주요 사용 예
    - 키패드 스캔 로직
    - 인터럽트 제어기



LAB : Specification



LAB : DUT

```
22  module interrupt_controller (
23      input [7:0] Int,        // 8 interrupt signal
24      input IntAck,           // Acknowledgement signal from CPU
25      output reg [2:0] IntID, // Index output from activate interrupt
26      output reg IntReq       // Valid intterupt request signal
27  );
28
29  reg [7:0] active_int;       // trace(store) activated interrupt
30
31  always @(*) begin
32      IntReq = 1'b0;          // default: no request
33      IntID = 3'b000;         // default: no output
34      active_int = Int;       // store current activated interrupt
35
36      if (IntAck) begin
37          active_int = 8'b0000_0000; // all request restore when CPU's approval
38      end else begin
39          casex (active_int)
40              8'b1xxx_xxxx: begin IntID = 3'b111; IntReq = 1'b1; end // Int7
41              8'b01xx_xxxx: begin IntID = 3'b110; IntReq = 1'b1; end // Int6
42              8'b001x_xxxx: begin IntID = 3'b101; IntReq = 1'b1; end // Int5
43              8'b0001_xxxx: begin IntID = 3'b100; IntReq = 1'b1; end // Int4
44              8'b0000_1xxx: begin IntID = 3'b011; IntReq = 1'b1; end // Int3
45              8'b0000_01xx: begin IntID = 3'b010; IntReq = 1'b1; end // Int2
46              8'b0000_001x: begin IntID = 3'b001; IntReq = 1'b1; end // Int1
47              8'b0000_0001: begin IntID = 3'b000; IntReq = 1'b1; end // Int0
48              default: begin IntID = 3'b000; IntReq = 1'b0; end      // no interrupt request
49          endcase
50      end
51  end
52
53  endmodule
```

LAB : Testbench

```
module tb_interrupt_controller;

    // signal declaration during simulation
    reg [7:0] Int;          // 8 interrupt input
    reg IntAck;             // Interrupt ack signal from CPU
    wire [2:0] IntID;       // Index output of activated interrupt
    wire IntReq;            // valid interrupt request signal

    // DUT instantiation
    interrupt_controller uut (
        .Int(Int),
        .IntAck(IntAck),
        .IntID(IntID),
        .IntReq(IntReq)
    );

    // Simulation
    initial begin
        // Monitoring: display simulation reasult as a text
        $monitor("Time=%0t | Int=%b | IntAck=%b | IntID=%b | IntReq=%b",
                 $time, Int, IntAck, IntID, IntReq);

        // INIT
        Int = 8'b0000_0000; // All interrupt deactivated
        IntAck = 1'b0;      // All ack signal deactivated

        #10 Int = 8'b0000_0001;  // Int0 activate(least priority)
        #10 Int = 8'b0010_0000;  // Int5 activate(mid priority)
        #10 Int = 8'b1000_0000;  // Int7 activat(highest priority)
        #10 IntAck = 1'b1;       // CPU's approval(ack) to interrupt : All request interrupt reset
        #10 IntAck = 1'b0;       // CPU's ack deactivated and wait for the next interrupt

        #10 Int = 8'b0100_0010;  // Complext input activate(Int6 selected)
        #10 IntAck = 1'b1;       // CPU's ack to Int6
        #10 IntAck = 1'b0;       // CPU's ack deactivated and wait for the next interrupt

        #10 Int = 8'b0000_0000;  // All input deactivated

        #20 $finish;             // Simulation finish
    end

endmodule
```

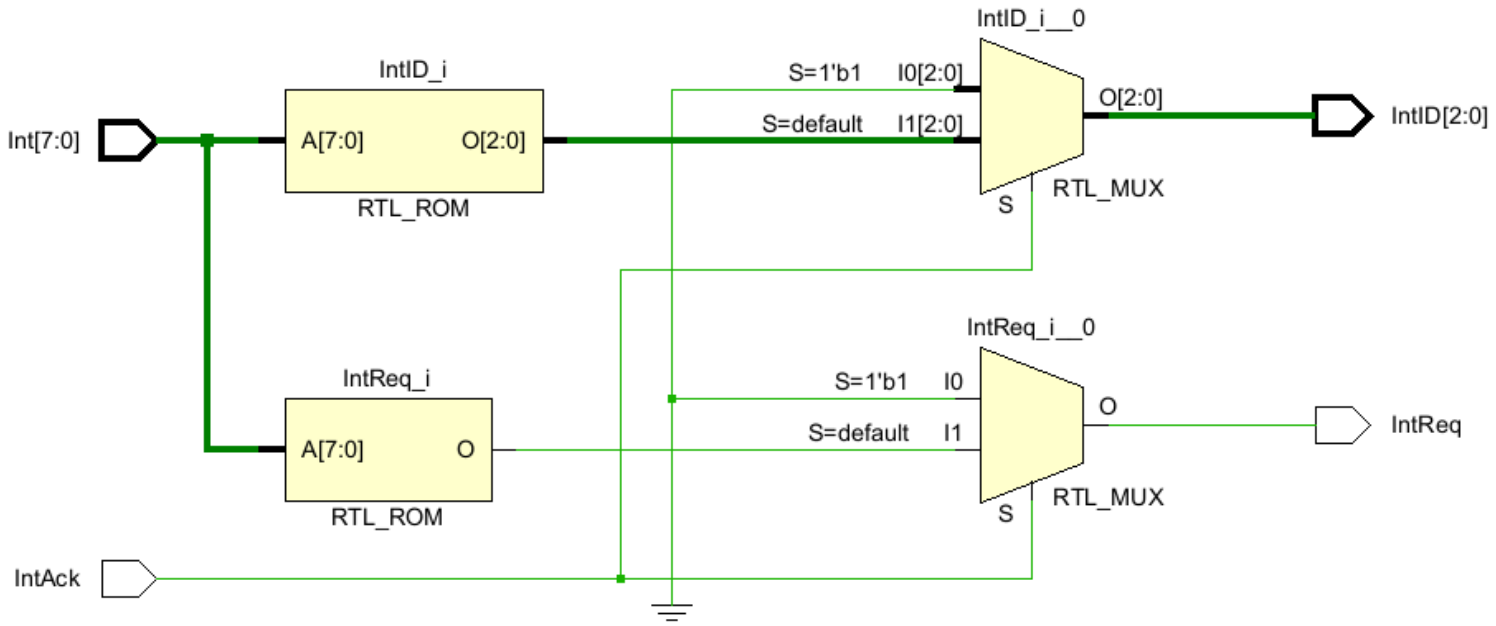
LAB : Encoder

- Wave Form

<note> 60ns에서 8'b0100_0010의 복합적인 인터럽트가 발생했지만, 우선순위 encoding에 따라 8'b01xx_xxxx로 해석되어 Int6으로 encoding됨.

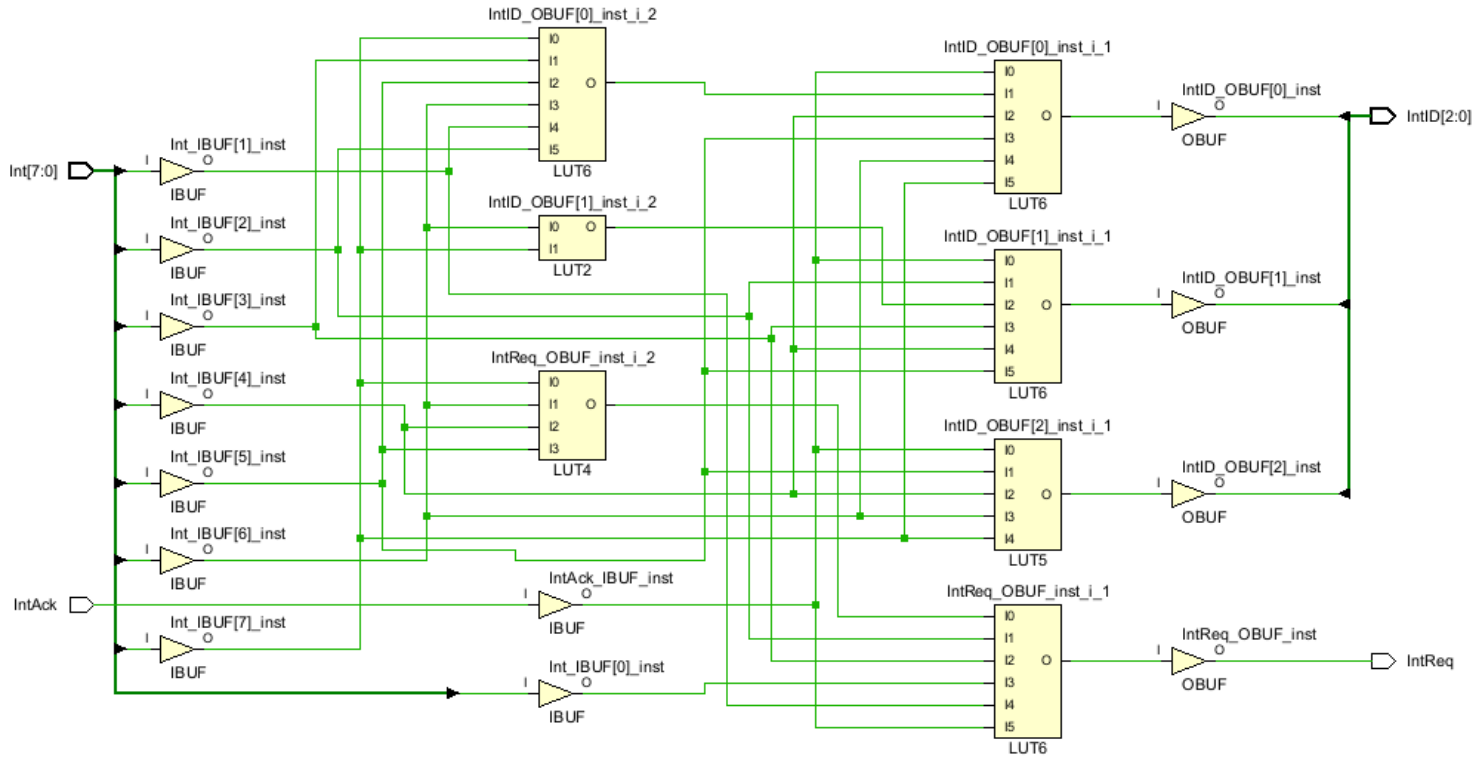
LAB : Encoder

- RTL Schematic



LAB : Encoder

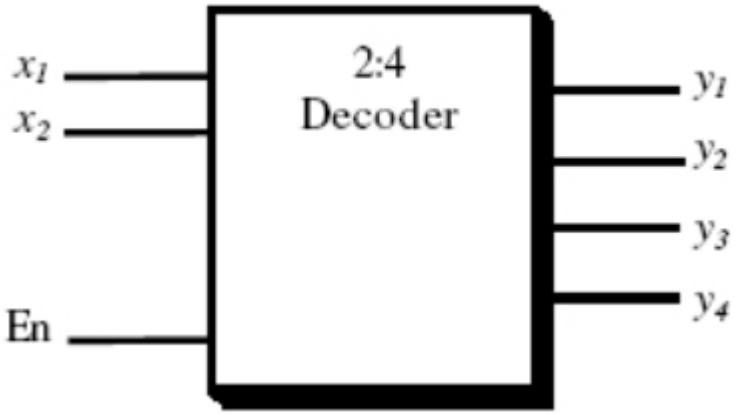
- Synthesized Schematic



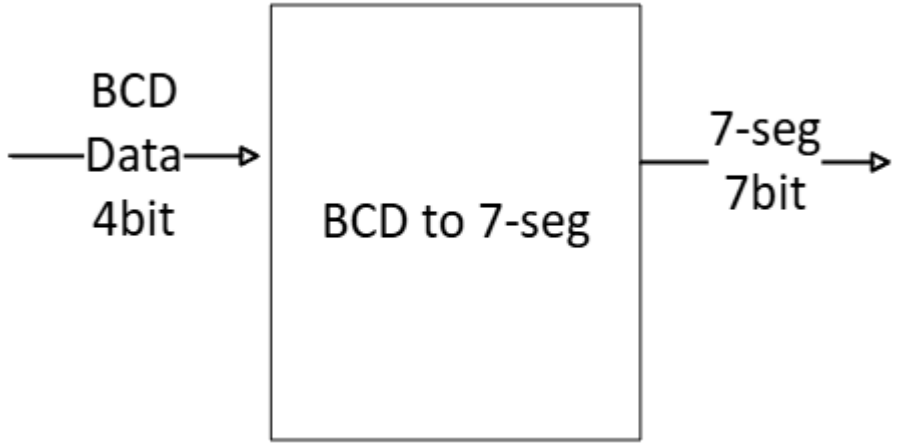
3. Decoder

Decoder

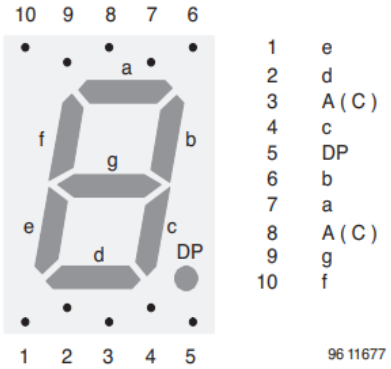
- 디코더 (Decoder)
  - 디코더는 인코더의 반대 기능을 수행하며, 코드화 된 입력을 원래의 형식으로 변환(코드화된 입력의 해석).
  - n개의 입력 라인을 2^n개의 출력 라인으로 변환합니다.
  - 입력 코드에 따라 하나의 출력만 활성화됩니다.
  - 주요 유형:
    - 2-to-4 디코더
    - 3-to-8 디코더
    - 4-to-16 디코더
    - BCD-to-7세그먼트 디코더

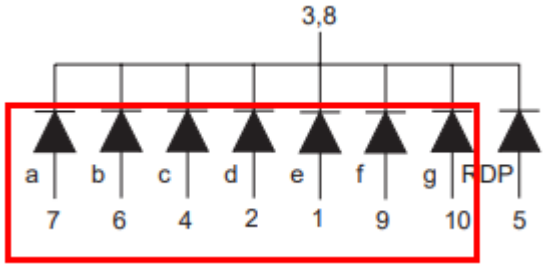


LAB : Specification



BCD Data(4bits)	7-segment(7bits)	display number
4'b0000	7'b1111110	0
4'b0001	7'b0110000	1
4'b0010	7'b1101101	2
4'b0011	7'b1111001	3
4'b0100	7'b0110011	4
4'b0101	7'b1011011	5
4'b0110	7'b1011111	6
4'b0111	7'b1110000	7
4'b1000	7'b1111111	8
4'b1001	7'b1111011	9
그 외(default)	7'b0000000	off





Source: VISHAY Semiconductor

LAB : DUT

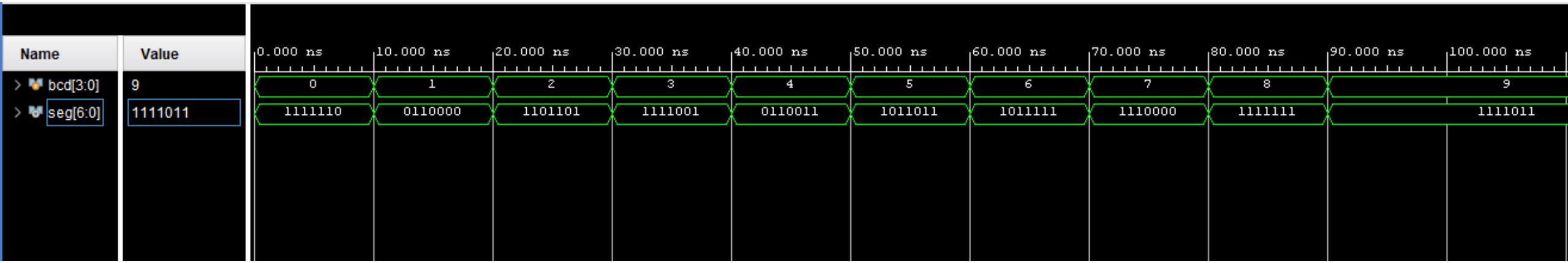
```
module bcd_to_7seg (
    input [3:0] bcd,        // 4-bit BCD(Binaray Coded Decimal) input
    output reg [6:0] seg    // 7-segment output
);

always @(*) begin
    case (bcd)
        4'b0000: seg = 7'b1111110; // 0
        4'b0001: seg = 7'b0110000; // 1
        4'b0010: seg = 7'b1101101; // 2
        4'b0011: seg = 7'b1111001; // 3
        4'b0100: seg = 7'b0110011; // 4
        4'b0101: seg = 7'b1011011; // 5
        4'b0110: seg = 7'b1011111; // 6
        4'b0111: seg = 7'b1110000; // 7
        4'b1000: seg = 7'b1111111; // 8
        4'b1001: seg = 7'b1111011; // 9
        default: seg = 7'b0000000; // wrong input -> deactivate segment
    endcase
end

endmodule
```

LAB : Testbench

```
module tb_bcd_to_7seg;

reg [3:0] bcd;          // BCD input signal
wire [6:0] seg;         // wired segment output

// DUT instantiation
bcd_to_7seg uut (
    .bcd(bcd),
    .seg(seg)
);

initial begin
    // Monitoring value change
    $monitor("Time=%0t | BCD Input=%b | Seven Segment Output=%b", $time, bcd, seg);

    // Apply stimulus
    bcd = 4'b0000; #10; // BCD: 0 -> Segment: a~g = "1111110"
    bcd = 4'b0001; #10; // BCD: 1 -> Segment: a~g = "0110000"
    bcd = 4'b0010; #10; // BCD: 2 -> Segment: a~g = "1101101"
    bcd = 4'b0011; #10; // BCD: 3 -> Segment: a~g = "1111001"
    bcd = 4'b0100; #10; // BCD: 4 -> Segment: a~g = "0110011"
    bcd = 4'b0101; #10; // BCD: 5 -> Segment: a~g = "1011011"
    bcd = 4'b0110; #10; // BCD: 6 -> Segment: a~g = "1011111"
    bcd = 4'b0111; #10; // BCD: 7 -> Segment: a~g = "1110000"
    bcd = 4'b1000; #10; // BCD: 8 -> Segment: a~g = "1111111"
    bcd = 4'b1001; #10; // BCD: 9 -> Segment: a~g = "1111011"

    $finish;
end

endmodule
```

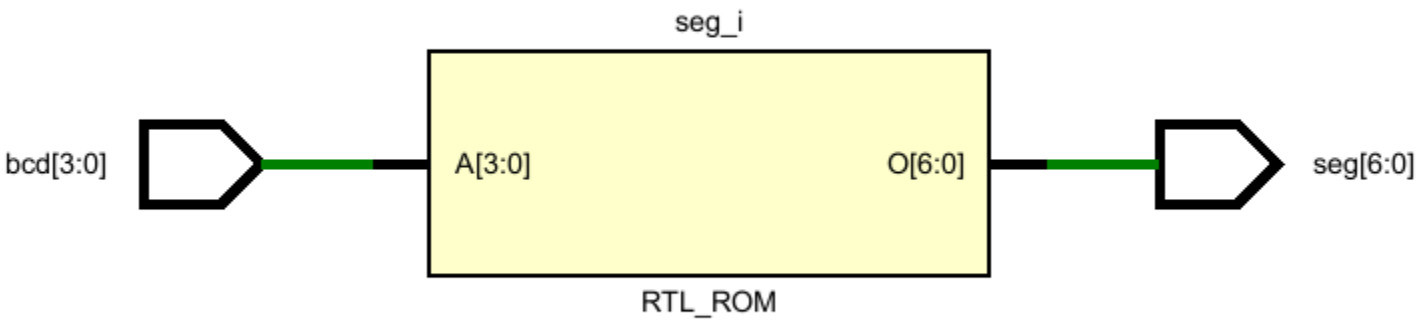
LAB : Decoder

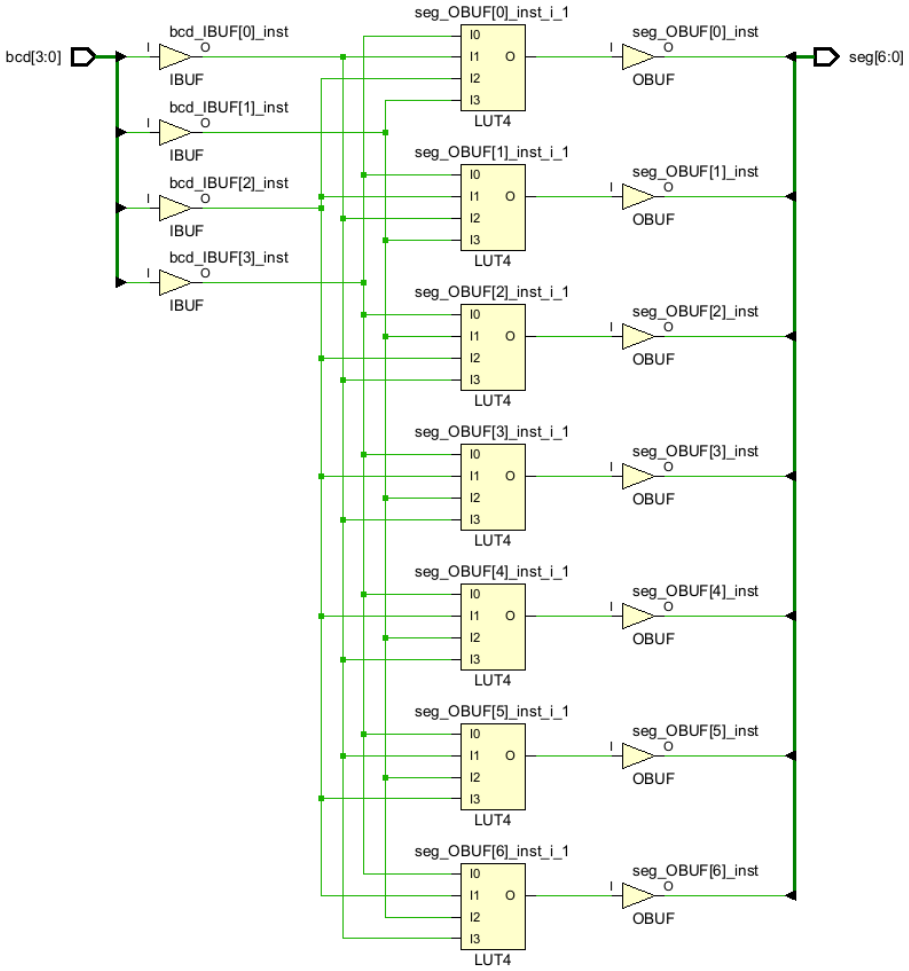
- Wave Form



LAB : Decoder

- RTL Schematic



LAB : Decoder

- Synthesized Schematic